Week 3: Designing Reusable Components

Designing and Building HitTastic! in React

This week we will contiue to build HitTastic!, in order to gain more practice with the concepts we've learnt so far and to consolidate what you learnt last year. In doing so, you will hopefully start to appreciate the benefit of reusable components.

Next week we will be introducing a range of new React features. However, to appreciate them, you need to gain a little more practical experience with basic React to identify a few problems that might arise.

You will also get the chance to analyse and design our small HitTastic! system. **Ideally you should do this as directed learning before the class, e.g. on Monday morning. We will then discuss your solutions in class, giving us more time to code the system.**

Designing the system

Have a look at this drawing of the intended UI of HitTastic!

Think about the "lifting up state" exercise from last time. By drawing a diagram, or explaining in words, state:

- What components are needed;
- What sub-components each component will need;
- What state variables each component will need;
- Using the principle of lifting up state, what props might each component need to receive from its parent?

If you draw a diagram using software (e.g. Paint), please **share it with me on OneDrive**. If explaining in words, please login and answer Exercise 1.

Discussion

Each area of the UI needs to be its own component. For example:

- Login component
- Search input component
- Search results component
- Add song component

The image below shows a more detailed analysis.

Coding Exercise 3.1: Coding the application

Once you've done this, you should start enhancing the functionality of your application by coding the system you have designed above. To do this:

- Clone this new Git repository located at:

  ```
  https://github.com/nwcourses/com623-wk3-starter.git
  ```

  This contains:
  - An Express server, setup to use `vite-express` already and with a range of endpoints including a login system;
  - An `index.html`, which will provide the page layout for your application;
  - A `main.jsx` and beginnings of an `App` component.
- Add the components that you developed last time, including Coding Exercise 2.4, to your `components` folder, replacing the basic `App`
- Code the remaining components you identified in the analysis and design exercise, above.